

React: TP2

Instruções:

- Para o TP2, devem ser entregues **as seguintes práticas das aulas 03 e 04**, que estão anexadas nos próximos slides;
 - Pré requisito: projeto na versão TP1;
- O valor total será de 10 (dez) pontos, com peso 2;
- Será **avaliado**:
 - Boas práticas e padronização de código, de acordo com o que foi apresentado em sala de aula;
 - Completude e funcionamento;
- Deve ser entregue o **link para github**;
 - Como a parte 02 é iterada em cima da parte 01, uma sugestão é organizar as entregas em **branches separadas**. Exemplo: react-tp1 e react-tp2.
- Para entregas fora do prazo, haverá desconto de nota proporcional ao tempo de atraso;
- Em caso de identificação de plágio, será atribuída nota 0 (zero);
- Em caso de dúvidas: discord ou email (bianka.maciел@icomp.ufam.edu.br)

Aula 03: Atividades práticas



(TP2) useEffect: Vamos praticar?



BootstrapClient

- Na aula passada, instalamos o CSS do Bootstrap;
- Para o Javascript, vamos utilizar o hook **useEffect**, da seguinte forma:
 - Na raiz da pasta de componentes, crie um novo arquivo nomeado "BootstrapClient.tsx";
 - No arquivo, crie e exporte um novo componente também com o mesmo nome "BootstrapClient", que retorne **null**, e seja client component;
 - Adicione um novo **useEffect**, com array de dependências vazio, cuja função seja requisitar o JS do Bootstrap:
 - `require("bootstrap/dist/js/bootstrap.bundle.min.js");`
 - No arquivo de **RootLayout** do projeto, importe o **BootstrapClient** no final dos componentes (abaixo de children e navbar)
- Verifique se o JS está funcionando corretamente, por meio do comportamento de colapsar da navbar em telas menores.

(TP2) useEffect: Vamos praticar?



BootstrapClient

```
'use client'
import { useEffect } from 'react'

export function BootstrapClient() {
  useEffect(() => {
    // eslint-disable-next-line @typescript-eslint/no-require-imports
    require('bootstrap/dist/js/bootstrap.bundle.min.js')
  }, [])
  return null
}
```

código aqui

```
export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode
}>) {
  return (
    <html
      lang='pt-br'
      className={` ${geistSans.variable} ${geistMono.variable}`}
    >
      <body>
        <Navbar />
        {children}
        <BootstrapClient />
      </body>
    </html>
  )
}
```

Axios no React: Vamos fazer o setup?



1. Instale o axios por meio do npm: **npm install axios**
2. Crie uma pasta dedicada para camada de serviços, em **/src/app/services**
3. Dentro da pasta, crie um arquivo para definição das apis, **api.ts**
4. Usando o axios, defina a URL para a nossa API de produtos, da seguinte forma:

```
wa-loja-react-next > src > app > services > TS api.ts > ...  
import axios from 'axios'  
  
export const productsApi = axios.create({  
  baseUrl: 'https://ranekapi.origamid.dev/json/api'  
})
```


React Query: Vamos fazer o setup?



Para **instalar** o react-query, basta seguir a documentação;

- npm i @tanstack/react-query

```
ReactQueryClient.tsx U X
src > app > components > ReactQueryClient.tsx > ...
1  "use client";
2
3  import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
4  import { useState } from "react";
5
6  export const ReactQueryClientProvider = ({
7    children,
8  }: {
9    children: React.ReactNode;
10 }) => {
11   const [queryClient] = useState(() => new QueryClient());
12   return (
13     <QueryClientProvider client={queryClient}>{children}</QueryClientProvider>
14   );
15 };
```

```
export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="pt-br">
      <body>
        <ReactQueryClientProvider>
          <Navbar />
          {children}
          <BootstrapClient />
        </ReactQueryClientProvider>
      </body>
    </html>
  );
}
```

código aqui

(TP2) useQuery: Vamos praticar?



Olhando para a página de **listagem de produtos**, faça as modificações necessárias para que o fetch ocorra por meio do axios e react-query, seguindo o passo a passo conforme apresentado nos slides anteriores.

- O endpoint de listagem de produtos é o seguinte:
<https://ranekapi.origamid.dev/json/api/produto>

(TP2) useMutation: Vamos praticar?



Para a funcionalidade de Favoritos, vamos precisar de um "banco de dados" para persistir os produtos favoritos. Para isso, vamos usar uma fake REST API por meio do json-server:

- Basta seguir o passo a passo desse repositório <https://github.com/biankavm/favorites-json-server>
 - que deve resultar em algo no formato
 - <https://favoritos-json-server-XXxx.vercel.app>
- E depois configurar no Axios uma entrada para esse outro endereço de API

```
import axios from 'axios'

export const productsApi = axios.create({
  baseURL: 'https://ranekapi.origamid.dev/json/api'
})

export const favoriteApi = axios.create({
  baseURL: 'https://shimmer-separate-blouse.glitch.me'
})
```

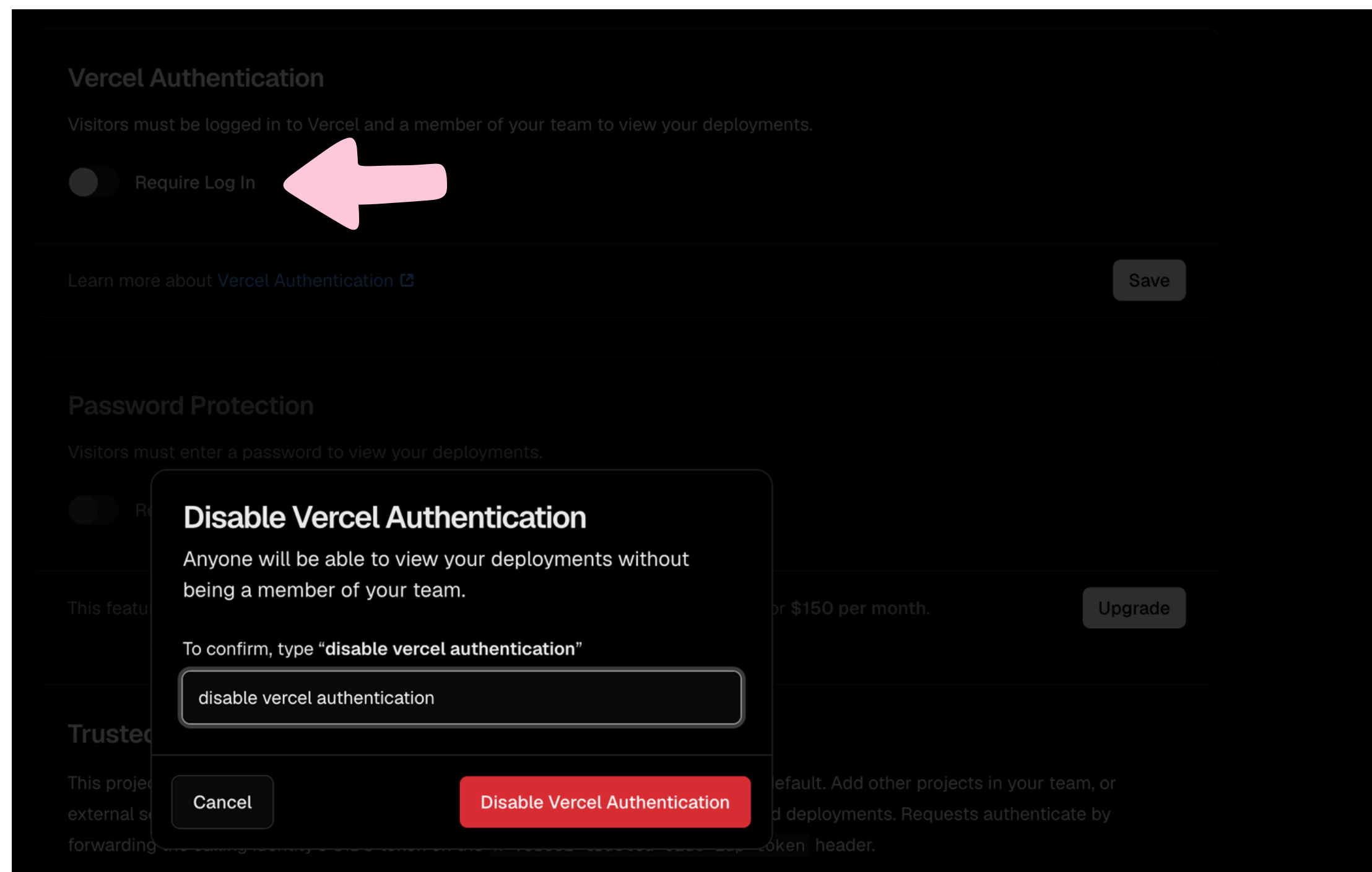
Aqui deverá ser o
endereço para a sua API



(TP2) useMutation: Vamos praticar?



Lembre-se de desabilitar a autenticação do vercel para se comunicar com a sua API:



(TP2) useMutation: Vamos praticar?



Agora olhando para a página de **listagem de produtos**, conforme os slides anteriores, faça as modificações necessárias para que:

1. Haja um botão de "favoritar" no **ProductCard**, e que o mesmo acione uma mutação para adicionar o produto alvo do clique em uma lista de favoritos
2. Para adicioná-lo, realize um **POST** no endpoint json-server de favoritos criado por você, utilizando a estrutura de axios + react query
3. Por meio dos helpers de "onSuccess" e "onError", implemente um feedback de sucesso ou erro para o usuário, da forma que preferir
 - a. Uma sugestão para facilitar a estilização é a utilização do pacote react-toastify - <https://www.npmjs.com/package/react-toastify>.

(TP2) useMutation: Vamos praticar?



Observações:

1. Caso opte pelo uso do **react-toastify**, deve-se importar o CSS no arquivo raiz de layout, similar ao que fizemos para o CSS do Bootstrap, para que funcione corretamente. **Documentação com exemplos:** <https://fkhadra.github.io/react-toastify/introduction/>

2. Para a API de favoritos, estamos utilizando o json-server, uma opção para simular o comportamento de APIs

a. Por isso, não haverá delay entre a resposta do "servidor". Caso queira simular um comportamento mais próximo da realidade, adicione um delay por meio de um **setTimeout**, como vimos no exemplo anterior.

```
react > wa-loja-react-next > src > app > layout.tsx > ...
1  import 'bootstrap/dist/css/bootstrap.min.css'
2  import type { Metadata } from 'next'
3  import { ToastContainer } from 'react-toastify'
4  import 'react-toastify/dist/ReactToastify.css'
5  import { Navbar } from './components/Navbar/Navbar'
6  import { BootstrapClient } from './components/client/BootstrapClient'
7  import { ReactQueryClientProvider } from './components/client/ReactQueryClient'
```

(TP2) useMutation: Vamos praticar?



Resultado esperado:

[Loja WA](#) [Início](#) [Carrinho](#) [Favoritos](#) [Sair](#)

Resumo do Carrinho

Quantidade total: 0

Valor total: R\$0.00

Produtos disponíveis:



Notebook
R\$ 2300

Adicionar no carrinho

Favoritar



Smartphone
R\$ 2399

Adicionar no carrinho

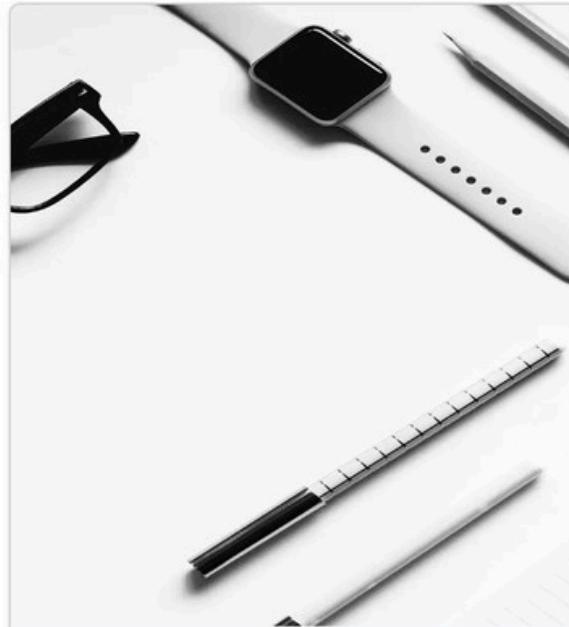
Favoritar



Câmera
R\$ 2199

Adicionar no carrinho

Favoritar



Smartwatch
R\$ 1199

Adicionar no carrinho

Favoritar

[veja aqui o gif](#)

Aula 04: Atividades práticas

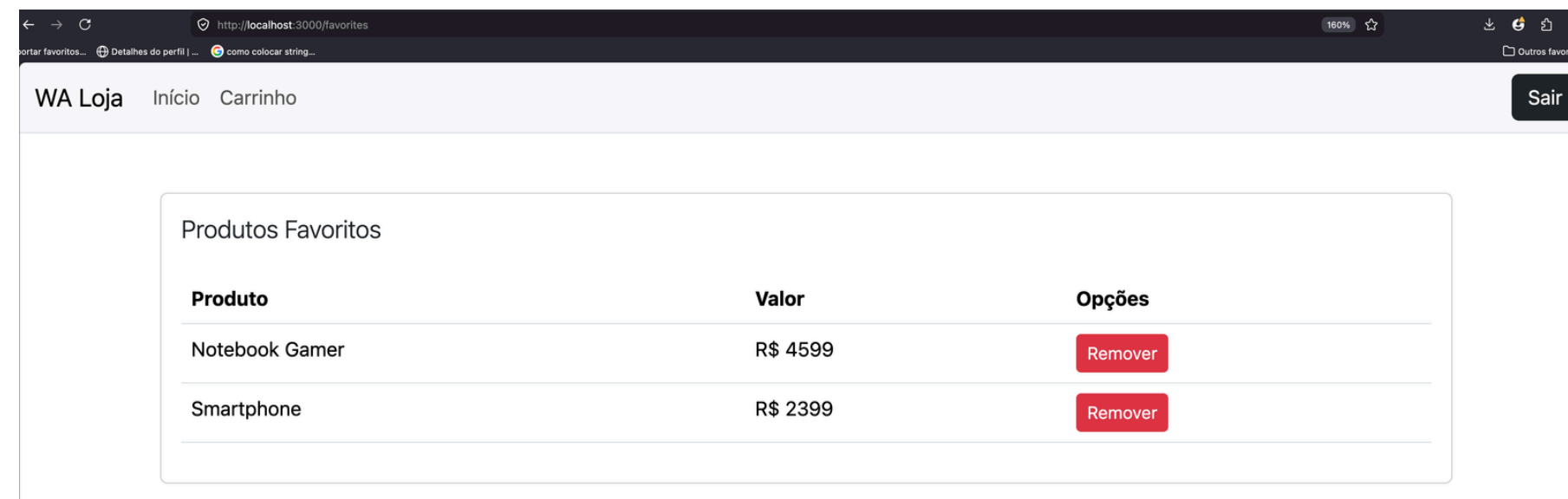


(TP2) useQuery e useMutation: Vamos praticar?



Agora, vamos criar uma nova página em nosso projeto para gerenciar a lista de produtos favoritos, sendo os requisitos:

1. Crie uma nova página na rota */favorites*;
2. Estruture um componente React para a página nova, contendo um título e a lista de produtos favoritos, com o botão de excluir em cada produto.
 - a. Faça bom uso do conceito de componetização.
3. Para renderizar a lista de produtos favoritos, implemente um **GET** no endpoint de favoritos criado por você na prática anterior, por meio do uso de axios + **useQuery** do React Query
 - a. Implemente uma interface para o estado de 'carregando' e 'erro'



Exemplo de layout que pode ser implementado para a lista de favoritos

```
interface Produto {  
  id: string;  
  fotos: Foto[];  
  nome: string;  
  preco: string;  
  descricao: string;  
  vendido: string;  
  usuario_id: string;  
}
```

A resposta da API será um array de objetos do tipo Produto

(TP2) useQuery e useMutation: Vamos praticar?



← → ↺

http://localhost:3000/favorites

160% ☆

⬇️ 🔍 📄 ☰

portar favoritos... Detalhes do perfil | ... como colocar string...

Outros favoritos

WA Loja

Início Carrinho

Sair

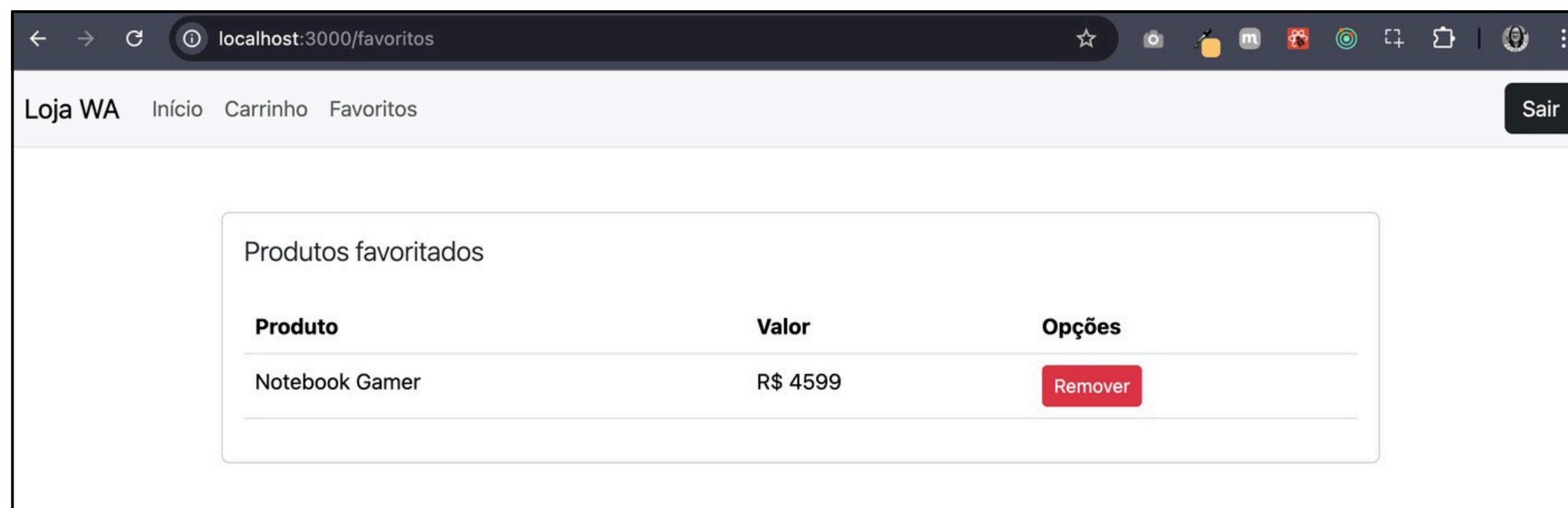
Produtos Favoritos

Produto	Valor	Opções
Notebook Gamer	R\$ 4599	Remover
Smartphone	R\$ 2399	Remover

(TP2) useQuery e useMutation: Vamos praticar?



1. Para o clique no botão de excluir de um determinado produto, implemente um **DELETE** no mesmo endpoint, por meio do uso de axios + **useMutation** do React Query
 - a. A API utilizada é do tipo REST, então o endpoint deve apontar para **`https://XXX.vercel.app/favoritos/{idParaSerDeletado}`**
 - b. Implemente um feedback de sucesso ou erro para o usuário, da forma que preferir
 - c. Uma sugestão para facilitar a estilização é a utilização do pacote **react-toastify** - <https://www.npmjs.com/package/react-toastify>



(TP2) useQuery e useMutation: Vamos praticar?



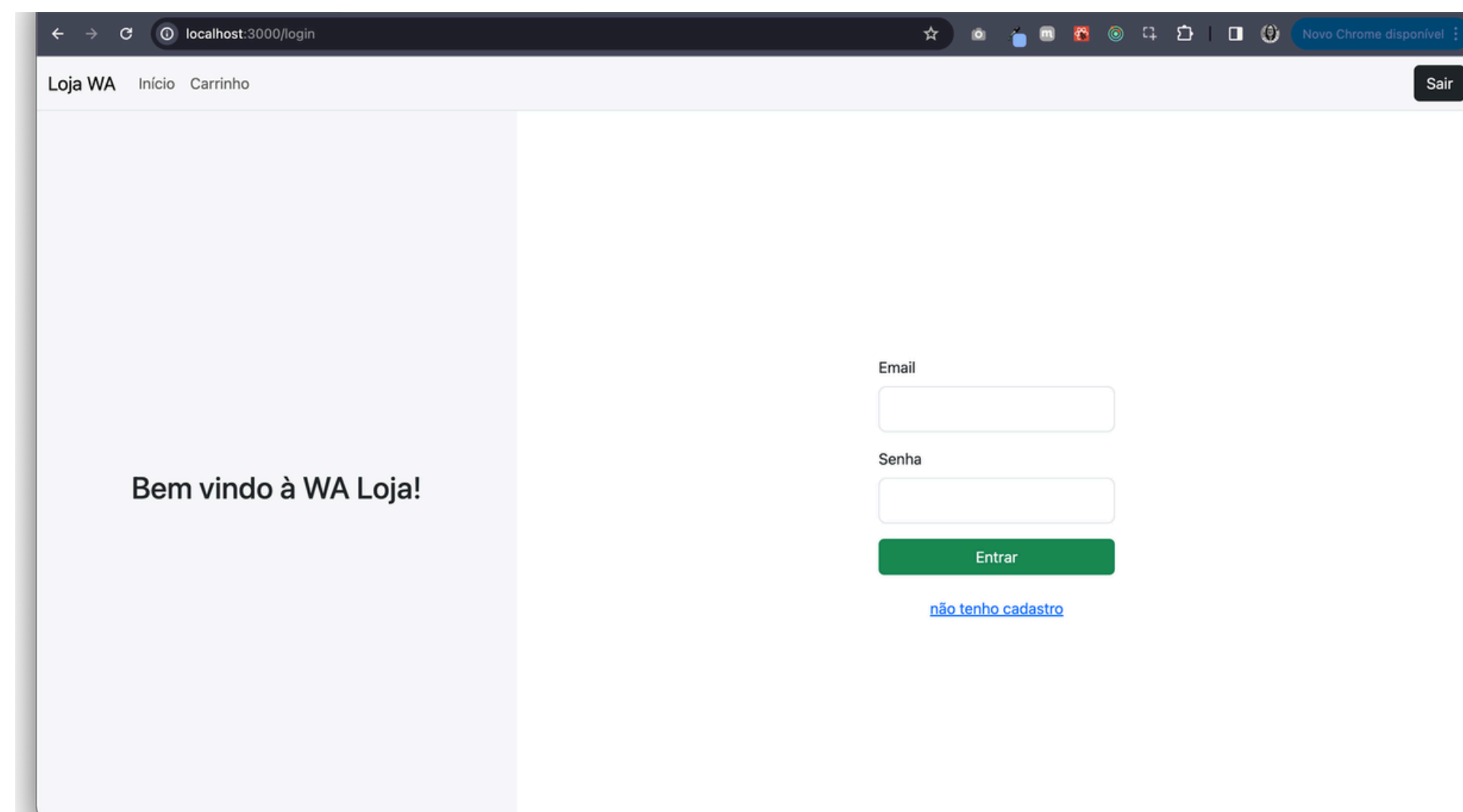
1. Note que ao deletar um produto da listagem de favoritos, a mesma ainda aparece de forma desatualizada. Para lidar com isso, implemente uma das duas formas mostradas anteriormente:
 - a. Uma delas é realizar o **refetch** da listagem de favoritos assim que a mutação der sucesso;
 - b. Outra é **invalidar** a query da listagem de favoritos, também no sucesso da mutação, o que também irá fazer com que haja um refetch. De ambas as formas, a listagem será atualizada.

(TP2) Hooks do Next: Vamos praticar?



Vamos agora adicionar novas páginas em nosso projeto e incrementar nossa **Navbar** com uma nova renderização condicional, baseada no **pathname**:

1. Crie duas novas páginas, com os códigos bootstrap iniciais abaixo:
 - a. página login -> código bootstrap [aqui](#)
 - b. página cadastro -> código bootstrap [aqui](#)
2. Note que ao navegar para essas novas páginas, a navbar continua aparecendo no topo. Contudo, não faz sentido exibí-la para usuários que ainda não estão logados, visto que possui função carrinho, botão sair, etc.

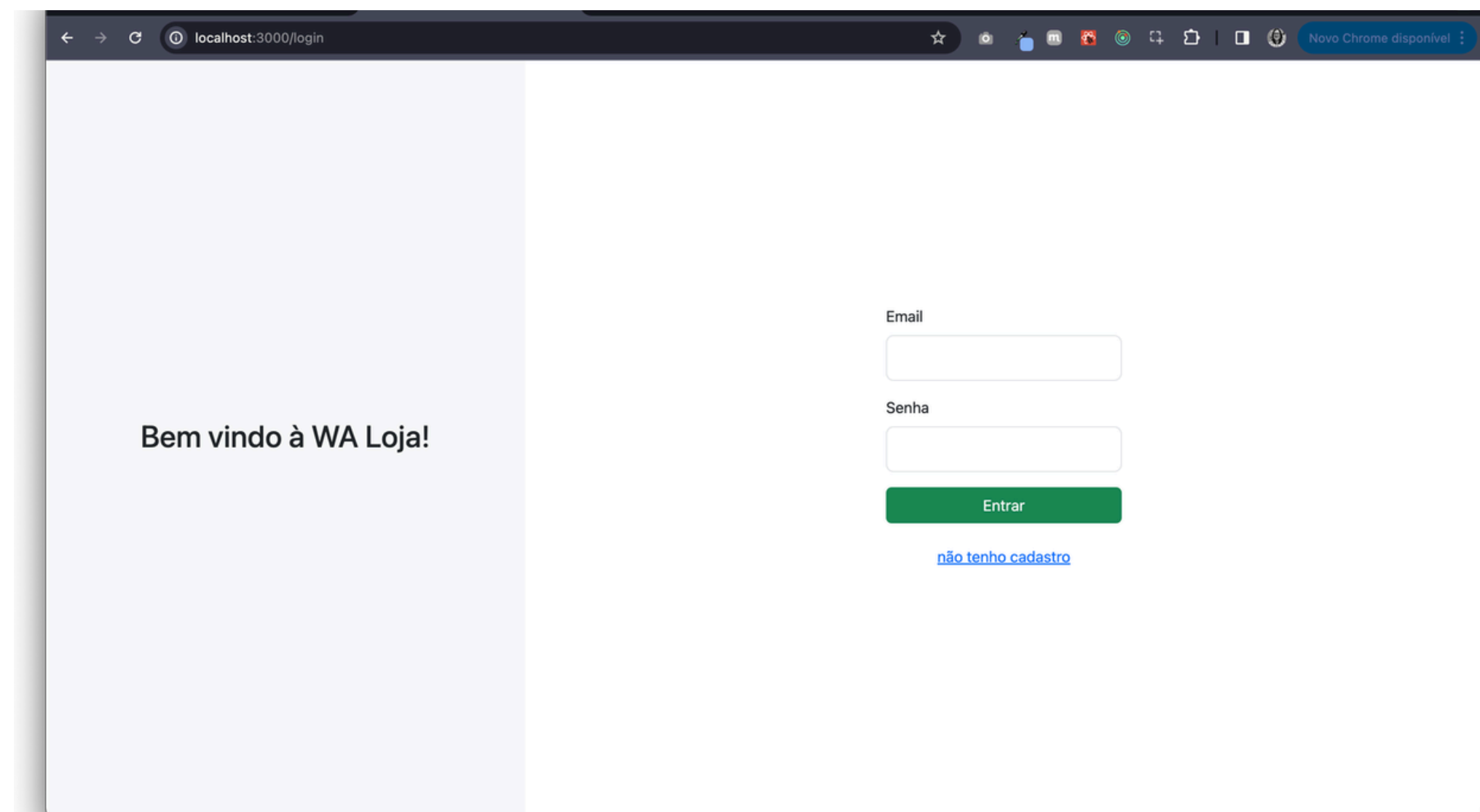


(TP2) Hooks do Next: Vamos praticar?



Por isso, adicione agora em **Navbar** um **retorno condicional**, da seguinte forma:

1. Utilize o hook `usePathname()` e confira em qual rota o usuário está atualmente;
2. Caso a rota seja a de `/login` ou `/register`, então retorne apenas `null`, fazendo então com que a navbar seja 'escondida';
3. Do contrário, retorne o componente de navbar normalmente.



Resultado esperado em `/login` e `/register`

(TP2) Hooks do Next e rotas dinâmicas: Vamos praticar?

Agora olhando para nossa listagem de produtos, vamos adicionar uma nova funcionalidade:

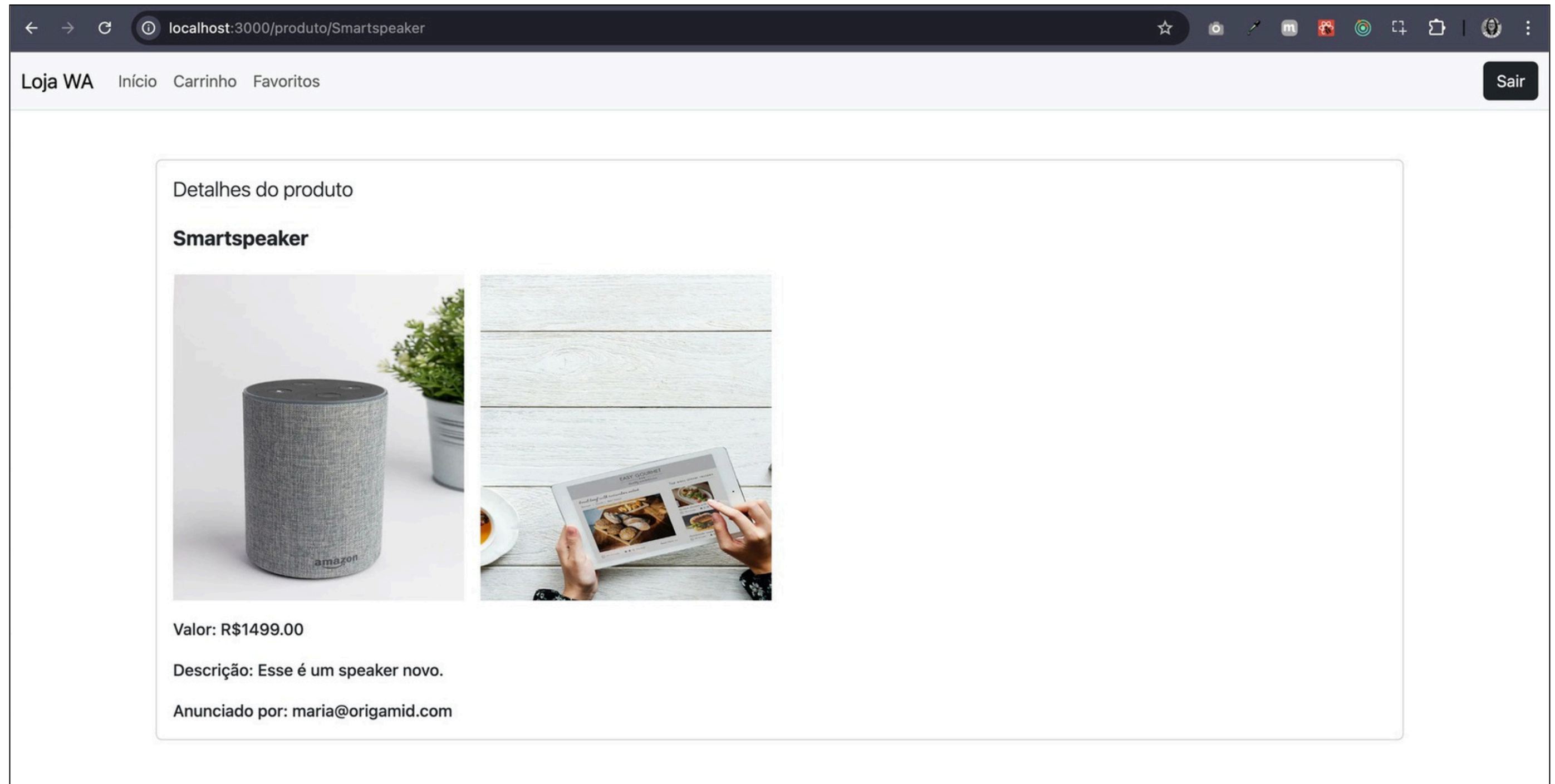
1. Crie uma nova página com rota dinâmica, com a seguinte configuração:
 - a. diretório: `app/product/[product]/page.tsx`
 - b. rota: `http://localhost:3000/product/notebook`
2. Na nova página, implemente um componente React client side **Produto**, com os requisitos a seguir:
 - a. Deve utilizar o hook `useParams()` para identificar o parâmetro 'produto' na url;
 - b. Deve utilizar o `useQuery` do React Query para realizar um fetch dos detalhes do produto, no seguinte endpoint: **`https://ranekapi.origamid.dev/json/api/produto/{nomeProduto}`**, seguindo os padrões de organização já vistos anteriormente.
 - c. O componente deve exibir em tela os detalhes do produto, quando houver um. Caso contrário, deve renderizar o estado de carregamento ou erro, se houver.
3. Acesse o código bootstrap da nova página [aqui](#)

(TP2) Hooks do Next e rotas dinâmicas: Vamos praticar?

1. De volta ao componente `ProductCard`, agora realize as seguintes alterações:
 - a. Na **imagem** do produto, adicione um `onClick` para uma função chamada `viewProductDetails`;
 - b. Implemente então a função, que deve receber como argumento o nome do produto e realizar um `router.push()` para a rota dinâmica, passando o **nome do produto** como parâmetro;
 - i. Utilize o hook `useRouter()` do `next/navigation`
2. Lembre-se de tipar corretamente o estado de `Product` e demais parâmetros

(TP2) Hooks do Next e rotas dinâmicas: Vamos praticar?

Resultado esperado:



[veja aqui o gif](#)

(TP2) Formulários no React: Vamos praticar?



Vamos implementar o controle dos nossos formulários de Login e Cadastro utilizando a abordagem de estado local padrão do React:

1. Nas páginas de `/login` e `/cadastro`, crie estados locais (`useState`) para armazenar os valores dos inputs (ex: email, senha, nome, etc.).
2. Transforme os inputs em componentes controlados: vincule os estados aos atributos `value` e adicione os eventos de `onChange` para atualizar esses estados.
3. Crie uma função (ex: `handleSubmit`) em cada página para lidar com a submissão do formulário.
4. Vincule essa função ao evento `onSubmit` da tag `<form>` e use `event.preventDefault()` para evitar o recarregamento padrão da página.
5. Dentro da função, faça um `console.log` com os dados digitados para confirmar que o estado está capturando tudo corretamente.
6. (Simulando o fluxo de entrada): Após o envio com sucesso, utilize o `useRouter` para redirecionar o usuário:
 - a. Tanto do `/login` quanto do `/cadastro` para a página inicial (`/`).
5. (Simulando o fluxo de saída): Lá no seu componente `Navbar`, crie uma função para o botão `Sair`. Essa função deve usar o `router.push()` para redirecionar o usuário de volta para a página de `/login`.

(TP2)(Opcional) React Hook Form: Vamos praticar?



Vamos refatorar nossos formulários das páginas de Login e Cadastro para fazer uso da biblioteca React Hook Form:

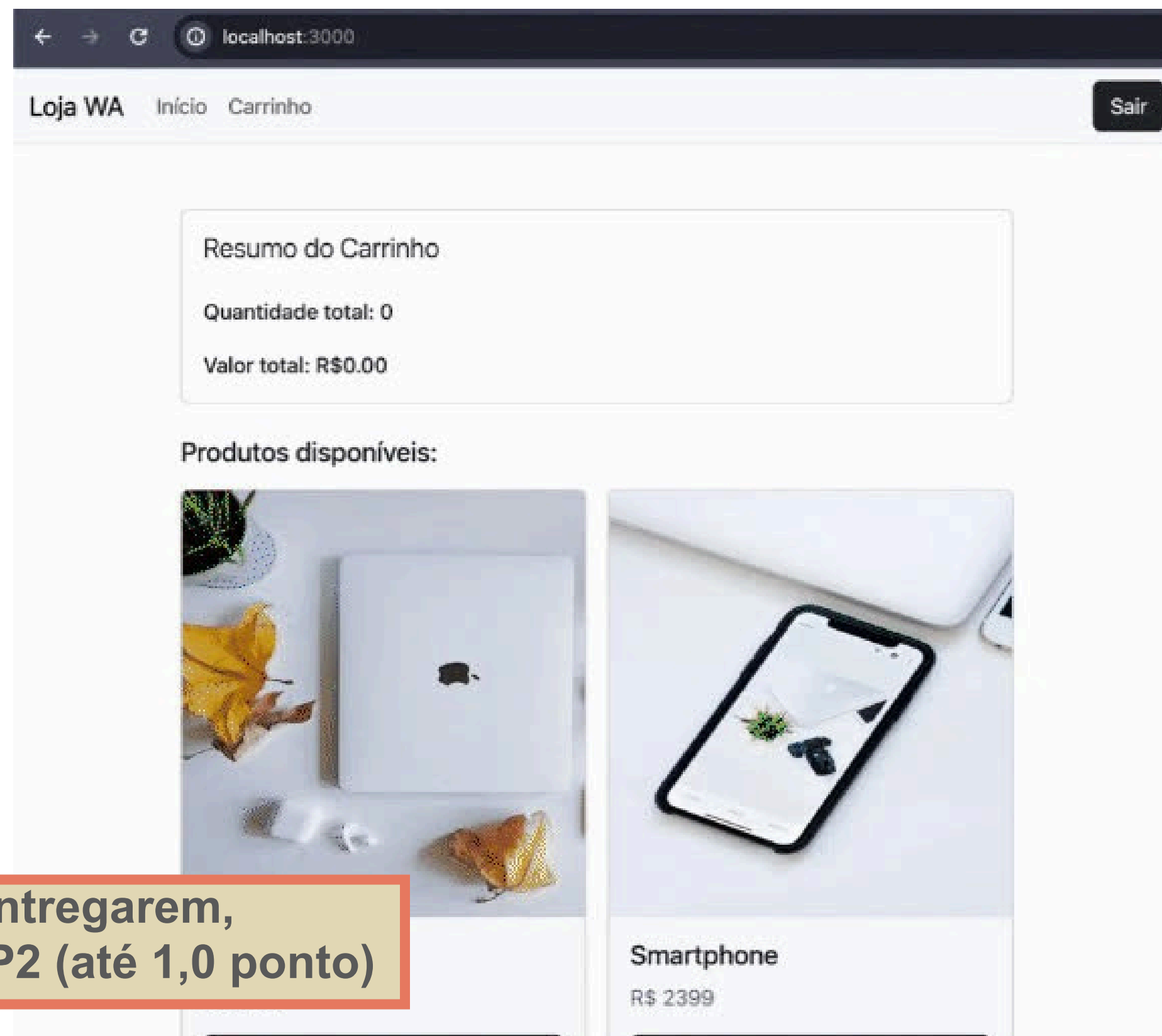
1. Nas duas páginas, remova os estados locais (useState) que você criou, assim como as propriedades value e onChange dos inputs.
2. Inicie o gerenciamento chamando o hook useForm().
3. Atenção: Não esqueça de criar uma interface (ou type) contendo os campos do seu formulário e passá-la para o hook (ex: useForm<FormularioLogin>()).
4. Desestruture do useForm as seguintes propriedades: a função register, a função handleSubmit e o objeto errors (que fica dentro de formState).
5. Utilize a função register nas tags <input> para conectar todos os campos ao formulário.
6. Adicione validações diretamente no register:
 - a. Para o email e demais campos, adicione a validação de "required".
 - b. Para a senha, adicione o "required" e também o "minLength" definido com o valor 6.
7. Feedback visual: Renderize uma mensagem de erro em uma tag logo abaixo de cada input. Essas mensagens só devem aparecer se houver um erro correspondente no objeto errors.
5. Por fim, no evento onSubmit da tag <form>, chame o handleSubmit do React Hook Form passando a sua função de envio como parâmetro. Após o sucesso, mantenha o redirecionamento para a página inicial (/) usando router.push().

(Opcional) React Hook Form: Vamos praticar?



Resultado esperado:

veja o gif aqui



- Esse exercício será opcional. Para os que entregarem, contará como pontuação extra no TP1 ou TP2 (até 1,0 ponto)